Python Data Science Libraries User Manual

Tableof Contents

1. Package Installation
2. Scikit-learn(sklearn)
3. VisualizationLibraries
    - Matplotlib
    - Seaborn
    - Plotly
4. Numerical ComputingLibraries
    - NumPy
    - Pandas
    - SciPy
5. HyperparameterTuning
    - Optuna
    - Hyperapt
    - Ray Tune

Package Installation

Installing Scikit-learn

pip install scikit-learn

Requirements: - Python (>= 3.10) - NumPy (>= 1.24.1)- SciPy (>= 1.10.0)

Installing Matplotlib, Seaborn, and Plotly

pip install matplotlib seaborn plotly

Installing NumPy, Pandas, andSciPy

pip install numpy pandas scipy

Alternative Installation (Using Conda):

conda install scikit-learn matplotlib seaborn plotly numpy pandas scipy

Scikit-learn (sklearn)

Purpose: Scikit-learnisanopen-sourcemachinelearning librarythatprovides simpleandefficienttoolsfordata mining anddata analysis.Itoffersimplemen tationsofmachinelearning algorithms for classification, regression,clustering, dimensionality reduction, model selection, and preprocessing.

Subpackages

1. sklearn.base Purpose: Base classes and utility functions for all estima tors in scikit-learn.

Key Classes: - BaseEstimator: Base class for all estimators in scikit-learn - ClassifierMixin: Mixin class for all classifiers - RegressorMixin: Mixin class for all regression estimators - TransformerMixin: Mixin class for all transform ers

Key Functions: - clone(): Construct a new unfitted estimator with the same parameters - is_classifier(): Return True if the given estimator is a classifier - is_regressor(): Return True if the given estimator is a regressor

2. sklearn.calibration Purpose: Probability calibration of classifiers to improve probability estimates.

Key Classes: - CalibratedClassifierCV: Probability calibration with iso tonic regression or logistic regression

Details: This module uses cross-validation to calibrate a classifier's probability outputs.It canuseeither sigmoid(Platt scaling)or isotonic regressionmethods to map raw classifier outputs to calibrated probabilities in the [0, 1] range.

3. sklearn.cluster Purpose: Clustering algorithms for grouping similar data points together.

Key Classes:-KMeans:K-Meansclusteringalgorithm-DBSCAN:Density-Based Spatial Clustering of Applications with Noise

Details: Clustering algorithms partition data into groups where similar ob servations are grouped together. KMeans is a centroid-based algorithm while DBSCAN is a density-based algorithm that can findarbitrarily shaped clusters.

4. sklearn.compose Purpose: Compositeestimatorsforbuildingcomplex machine learning pipelines.

Key Classes: -ColumnTransformer: Apply transformers todifferent columns of arrays or DataFrames

Details: This module allows you to apply different preprocessing and trans formation steps to different columns of your dataset. For example, you can standardize numerical features while one-hot encoding categorical features.

5. sklearn.covariance Purpose: Covariance estimation methods.

Key Classes: - EmpiricalCovariance: Maximum likelihood covariance esti

mator

Details: Usedfor estimating thecovariancematrixoffeatures, whichisuseful for understanding relationships between variables and for anomaly detection.

6. sklearn.cross_decomposition Purpose: Crossdecompositionalgo rithms for relating two datasets.

Key Classes: - PLSRegression: Partial Least Squares regression

Details: PLS regressionisusefulwhenyouhavetwoblocksof variablesandyou want to find the latent variables that best explain both blocks simultaneously.

7. sklearn.datasets Purpose: Utilities to load and generate datasets.

Key Functions: - load_iris(): Load the Iris dataset - make_classification(): Generate a random classification dataset

Details: Provides access to toy datasets for learning and testing, as well as utilities to generate synthetic datasets with specific properties.

8. sklearn.decomposition Purpose: Matrixdecompositionalgorithmsfor dimensionality reduction.

Key Classes: -PCA:PrincipalComponentAnalysis-NMF:Non-NegativeMatrix Factorization

Details: PCA transforms data to a new coordinate system where the greatest variance lies on the first coordinate (firstprincipal component). NMF factorizes a matrix into two non-negative matrices, useful for parts-basedrepresentation.

9. sklearn.discriminant_analysis Purpose: Discriminantanalysisclassi fiers.

Key Classes: - LinearDiscriminantAnalysis: Linear Discriminant Analysis classifier

Details: LDAisa classifierwitha lineardecisionboundary, generatedbyfitting class conditional densities to the data and using Bayes' rule. It also performs dimensionality reduction.

10. sklearn.dummy Purpose: Dummy estimators for baseline compar isons.

Key Classes: - DummyClassifier: Makes predictions using simple rules (e.g., always predict the most frequent class)

Details: Useful for establishing baseline performancethatmore sophisticated models should exceed.

11. sklearn.ensemble Purpose: Ensemble-basedmethodsfor classification, regression, and anomaly detection.
Key Classes: - RandomForestClassifier: Random forest classifier using mul tiple decision trees - GradientBoostingClassifier: Gradient boosting classi fier

Details: Ensemble methods combine multiple models to improve robustness and accuracy. Random forests use bagging (bootstrap aggregating) while gra dient boosting builds trees sequentially to correct errors of previous trees.

12. sklearn.exceptions Purpose: Customexceptions for scikit-learn.

Key Classes: - NotFittedError: Exception raised when using an unfitted estimator

Details: Providesspecific exceptiontypesforbetter errorhandlinginmachine learning workflows.

13. sklearn.experimental Purpose: Experimental features that may change in future releases.

Key Modules: - enable_iterative_imputer: Enables iterative imputation for missing values

Details: Contains features that are still being tested and refined before becom ing part of the stable API.

14. sklearn.feature_extraction Purpose: Feature extraction from raw data.

Key Classes: - CountVectorizer: Convert text documents to a matrix of token counts

Details: Useful for converting text data into numerical features for machine learning algorithms.

15. sklearn.feature_selection Purpose: Feature selection algorithms to select relevant features.

Key Classes: - SelectKBest: Select features according to the k highest scores

Details: Helps reduce overfitting, improve accuracy, and reduce training time by selecting the most relevant features.

16. sklearn.gaussian_process Purpose: Gaussian process regression and classification.

Key Classes: - GaussianProcessRegressor: Gaussian process regression

Details: Gaussian processes provide probabilistic predictions and can

capture uncertainty in predictions. Useful when you need confidence intervals.

17. sklearn.impute Purpose: Strategies for imputingmissing values.

Key Classes: - SimpleImputer: Univariate imputer for completing missing values

Details: Provides strategies like mean, median, most frequent, or constant value imputation for handling missing data.

18. sklearn.inspection Purpose: Modelinspection andunderstanding.

Key Functions: - permutation_importance(): Permutation-based feature importance

Details: Helps understand which features are most important for model pre dictions.

19. sklearn.isotonic Purpose: Isotonic regression.

Key Classes: - IsotonicRegression: Isotonic (monotonic) regression

Details: Fits a non-decreasing function to data, useful for calibration and monotonic transformations.

20. sklearn.kernel_approximation Purpose: Approximations of kernel functions.

Key Classes:-RBFSampler:Approximatea RBFkernelusing randomFourier features

Details: Provides scalable alternatives to exact kernel methods, enabling kernel like methods on large datasets.

21. sklearn.kernel_ridge Purpose: Kernel ridge regression.

Key Classes: - KernelRidge: Kernel ridge regression

Details: Combines ridge regression withthe kernel trick, allowingnon-linear relationships to be learned.

22. sklearn.linear_model Purpose: Generalizedlinear models for regres sion and classification.

Key Classes: - LinearRegression: Ordinary least squares linear regression - LogisticRegression: Logistic regression classifier

Details: Linear regression minimizes the sum of squared residuals. Logistic regression uses a logistic function to model binary outcomes.

23. sklearn.manifold Purpose: Manifold learning and non-linear dimen sionality reduction.

Key Classes:- TSNE:t-distributedStochasticNeighbor Embedding

Details: t-SNE is particularly good for visualizing high-dimensional data by reducing it to 2D or 3D while preserving local structure.

24. sklearn.metrics Purpose: Metrics for evaluating model performance.

Key Functions: - accuracy_score(): Accuracy classification score - silhouette_score(): Compute the mean silhouette coefficient of all samples

Details: Providescomprehensivemetricsfor classification(accuracy,precision, recall, F1), regression (MSE, R^2), andclustering (silhouette score, adjusted rand index).

25. sklearn.mixture Purpose: Gaussianmixture models.

Key Classes: - GaussianMixture: Gaussian mixture model

Details: Probabilistic model assuming data is generated from a mixture of Gaussian distributions. Useful for soft clustering and density estimation.

26. sklearn.model_selection Purpose: Model selection and evaluation tools.

Key Functions: - train_test_split(): Split arrays into random train and test subsets - cross_val_score(): Evaluate a score by cross-validation

Details: Providestoolsfor splittingdata, cross-validation,hyperparameter tun ingwithGridSearchCVandRandomizedSearchCV,andlearningcurveanalysis.

27. sklearn.multiclass Purpose: Multiclass and multilabel classification strategies.

Key Classes: - OneVsRestClassifier: One-vs-the-rest (OvR) multiclass strategy

Details: Extends binary classifiers to handle multiclass problems by training one classifier per class.

28. sklearn.multioutput Purpose: Multioutput regression andclassifica tion.

Key Classes: - MultiOutputRegressor: Multi-target regression

Details: Fits one regressor per target, useful when predicting multiple contin uous values simultaneously.

29. sklearn.naive_bayes Purpose: Naive Bayes

classifiers. **Key Classes:** - GaussianNB: Gaussian Naive Bayes

classifier

Details: Based on Bayes' theorem with the "naive" assumption of indepen dencebetweenfeatures. Fast andeffectivefor text classificationandotherhigh dimensional problems.

30. sklearn.neighbors Purpose: Nearest neighbors algorithms.

Key Classes: - KNeighborsClassifier: K-nearest neighbors classifier

Details: Classifiesbasedonthe majorityclass among the k nearest neighbors. Non-parametric and can learn complex decision boundaries.

31. sklearn.neural_network Purpose: Neural network

models. **Key Classes:** - MLPClassifier: Multi-layer Perceptron

classifier

Details: Feedforward artificial neural network with backpropagation. Can learn non-linear models and perform both binary and multiclass classification.

32. sklearn.pipeline Purpose: Pipeline construction utilities.

Key Classes: - Pipeline: Pipeline of transforms with a final estimator - make_pipeline(): Construct a Pipeline from the given estimators

Details: Pipelineshelpstreamlinemachinelearningworkflowsbychaining pre processing stepsanda final estimator. Thisensuresall stepsare appliedconsis tently during training and prediction.

33. sklearn.preprocessing Purpose: Data preprocessing utilities.

Key Classes: - StandardScaler: Standardize features by removing the mean and scaling to unit variance - LabelEncoder: Encode target labels with value between 0 and n_classes-1 - OneHotEncoder(): Encode categorical features as a one-hot numeric array - MinMaxScaler(): Scale features to a specified
range (default 0 to 1) - StandardScaler(): Standardize features by removing the mean and scaling to unit variance (z-score normalization) - normalize(): Normalize samples individually to unit norm (l1, l2, or max norm)

Details: StandardScaler transforms features to have zero mean and unit variance, improvingperformanceof manyalgorithms. LabelEncoder converts cate- gorical labels to integers. OneHotEncoder can be fitted to training data and reused to transform new inputs, supporting unknown categories and sparse matrix output. This transformation is essential for converting non numeric (categorical) data into a numeric format required by many machine learning models. OneHotEncoder can be fitted to training data and reused to transform new inputs, supporting unknown categories and sparse matrix output. This transformation is essential for converting non-numeric (categorical) data into a numeric format required by many machine learning

models.

34. sklearn.random_projection Purpose: Random projection for dimen sionality reduction.

Key Classes: - GaussianRandomProjection: Reduce dimensionality through Gaussian random projection

Details: Fast dimensionality reduction technique that preserves distances ap proximately, based on the Johnson-Lindenstrauss lemma.

35. sklearn.semi_supervised Purpose: Semi-supervised learning algo rithms.

Key Classes: - LabelPropagation: Semi-supervised label propagation

Details: Uses graph-based methods to propagate labels from labeled to unla beled data points based on similarity.

36. sklearn.svm Purpose: Support VectorMachines.

Key Classes:- SVC: Support Vector Classification

Details: SVMs find the hyperplane thatbest separates classes with maximum margin.Effectiveinhigh-dimensional spacesandcanusedifferentkernelfunc tions for non-linear decision boundaries.

37. sklearn.tree Purpose: Decisiontree algorithms.

Key Classes: - DecisionTreeClassifier: Decision tree classifier

Details: Builds a tree structure where each internal node represents a test on a feature, eachbranchrepresentstheoutcome, andeachleaf representsa class label.

38. sklearn.utils Purpose: Utility functions.

Key Functions: - check_array(): Input validation on an array, list, sparse matrix or similar - shuffle(): Shuffle arrays in a consistent way

Details: Provideshelper functionsforvalidation,arraymanipulation,andother common operations.

Visualization Libraries

Matplotlib

Purpose: Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a MATLAB-like interface for plotting and is the foundation for many other plotting libraries.

Core Modules

1. matplotlib.pyplot Purpose: State-basedinterfacefor matplotlib (simi lar to MATLAB).

Key Functions: - plot(): Plot y versus x as lines and/or markers - scatter(): Create a scatter plot - hist(): Plot a histogram - bar(): Make a bar plot - show(): Display all open figures - savefig(): Save the current figure

Details: pyplot provides a convenient interface for quickly creating plots. It maintains state (current figure and axes) and provides functions that operate on the current axes.

2. matplotlib.axes Purpose: TheAxesclassrepresents one (sub-)plotin a figure.

Key Attributes: - Contains plotted data, axis ticks, labels, title, legend

Key Methods: - plot(), scatter(), hist(), bar(): Plotting methods - set_xlabel(), set_ylabel(), set_title(): Set labels and title - legend(): Place a legend on the axes - grid(): Configure the grid lines

Details: Axes is the main object for creating plots and contains most plotting methods. It represents the actual plot area within a figure.

3. matplotlib.figure Purpose: Top-level container forall plot elements.

Key Attributes: - Contains one or more Axes (subplots) - Figure size, DPI, background color

Key Methods: - add_subplot(): Add a subplot to the figure - add_axes():

Add an axes at a specific position - savefig(): Save the figure

Details: Figure is the top-level container that holds everything you see in a plot window. It contains one or more Axes and other elements like titles and legends.

4. matplotlib.patches Purpose: Classes for drawing 2D

shapes. **Key Classes:** - Rectangle, Circle, Polygon, Arrow

Details: Used to add geometric shapes to plots for annotations or emphasis.

5. matplotlib.path Purpose: Path operations for creating customshapes.

Details: Provides low-levelpath operations used internally for drawing shapes and creating custom artists.

6. matplotlib.lines Purpose: Line2D class for drawing lines.

Details: Represents line segments in 2D space with properties like color, linewidth, and linestyle.

7. matplotlib.text Purpose: Text class for adding text to plots.

Key Classes: - Text: Text object for annotations - Annotation: Annotated text with optional arrows

Details: Usedforadding labels,titles, annotations,andother text elements to plots.

8. matplotlib.collections Purpose: Classesfor efficientdrawingof groups of objects.

Details: Collections allow efficient rendering of multiple similar objects (e.g., scatter plot markers, line segments).

9. matplotlib.colors Purpose: Color specifications and colormapping.

Key Classes: - Colormap: Base class for color maps - Normalize: Normalize data into [0, 1] interval

Details: Providesutilitiesfor color specification, colormaps, andcolornormal ization for scalar data visualization.

10. matplotlib.cm Purpose: Colormapmodule.

Details: Containspre-definedcolormaps(viridis,plasma, jet, etc.)formapping scalar data to colors.

11. matplotlib.ticker Purpose: Tick locating andformatting.

Key Classes: - Locator: Base class for tick locators - Formatter: Base class for tick formatters

Details: Controls where ticks are placed on axes and how they are formatted

(e.g., scientific notation, dates, currency).

12. matplotlib.scale Purpose: Axis scaling (linear, log, etc.).

Key Classes: - LinearScale, LogScale, SymmetricalLogScale

Details: Defines how data is scaled on axes, enabling logarithmic, symlog, and other non-linear scales.

13. matplotlib.transforms Purpose: Coordinate transformations.

Details: Handles transformations between different coordinate systems (data, axes, figure, display).

14. matplotlib.artist Purpose: Base class for objects that render into a FigureCanvas.

Details: Everything visible on the figure is an Artist (lines, text, images, axes, figure itself).

15. matplotlib.backend_bases Purpose: Abstract base classes for ren dering backends.

Details: Defines the interface for different rendering backends (GUI toolkits, file formats).

16. matplotlib.backends Purpose: Rendering backends for different out put formats and GUI toolkits.

Details: Provides backends for rendering to different formats (PDF, SVG, PNG) and GUI frameworks (Qt, Tk, WX).

17. matplotlib.offsetbox Purpose: Classes for offset boxes (annotations with boxes).

Details: Used for creating complex annotations with backgrounds, borders, and multiple text/image elements.

18. matplotlib.container Purpose: Container classesforaggregateartists.

Details: Groups related artists together (e.g., bar containers, error bar con tainers).

19. matplotlib.gridspec Purpose: Grid layout for subplots.

Key Classes: - GridSpec: Specifies the geometry of subplots

Details: Allows fine-grained control over subplot layout, including different sized subplots in a grid.

20. matplotlib.animation Purpose: Animation framework.

Key Classes: - FuncAnimation: Animation by repeatedly calling a function - ArtistAnimation: Animation using a fixed set of Artist objects

Details: Provides tools for creating animated plots by updating plot elements over time.

21. matplotlib.widgets Purpose: Interactive GUI widgets.

Key Classes: - Slider, Button, RadioButtons, CheckButtons

Details: Adds interactive elements to matplotlib plots for parameter explo ration and data interaction.

Seaborn

Purpose: Seabornisa statisticaldata visualizationlibrarybuiltonmatplotlib. It provides a high-level interface for drawing attractive and informative statis tical graphics.

Core Modules

1. seaborn.relational Purpose: Visualizing statistical relationships be tween variables.

Key Functions: - scatterplot(): Draw a scatter plot with possibility of several semantic groupings - lineplot(): Draw a line plot with possibility of several semantic groupings - relplot(): Figure-level interface for drawing relational plots

Details: Usedfor exploring relationshipsbetweentwoormorevariables, with support for categorical groupings via hue, size, and style.

2. seaborn.distributional Purpose: Visualizingdistributions of datasets.

Key Functions: - histplot(): Plot univariate or bivariate histograms - kdeplot(): Plot univariate or bivariate distributions using kernel density estimation - ecdfplot(): Plot empirical cumulative distribution functions - rugplot(): Plot marginal distributions by drawing ticks along axes - displot(): Figure-level interface for drawing distribution plots

Details: Providesmultiplewaystovisualizedistributionsof data,fromsimple histograms to kernel density estimates and empirical CDFs.

3. seaborn.categorical Purpose: Visualizing categoricaldata.

Key Functions: - stripplot(): Draw a scatterplot with one categorical vari able- swarmplot(): Draw a categorical scatterplotwithnon-overlapping points - boxplot(): Draw a box plot to show distributions with respect to categories - violinplot(): Draw a combination of boxplot and kernel density estimate - pointplot(): Show point estimates and confidence intervals as rectangular bars - barplot(): Show point estimates and confidence intervals as rectangular bars - countplot(): Show the counts of observations in each categorical bin - catplot(): Figure-level interface for drawing categorical plots

Details: Provides various ways to visualize categorical data at differentlevels

of granularity, from individual points to statistical summaries.

4. seaborn.regression Purpose: Visualizing linear relationshipswithre gression models.

Key Functions: - regplot(): Plot data and a linear regression model fit - lmplot(): Plot data and regression model fits across a FacetGrid - residplot(): Plot the residuals of a linear regression

Details: Combines scatter plots with fitted regression lines, with automatic computation of confidence intervals.

5. seaborn.matrix Purpose: Visualizingmatrices and DataFrames.

KeyFunctions:-heatmap(): Plot rectangulardataasa color-encodedmatrix -

clustermap(): Plot a matrix dataset as a hierarchically-clustered heatmap

Details: Useful for visualizing correlation matrices, confusion matrices, and other tabular data with color encoding.

6. seaborn.figure Purpose: Figure-level grid interfaces.

Key Classes: - FacetGrid: Multi-plot grid for plotting conditional relation ships

Details: Provides infrastructure for creatingmultiple subplots based on cate gorical variables.

7. seaborn.objects Purpose: Object-orientedinterface fordeclarative vi sualization (newer API).

Details: Providesa grammar-of-graphicsstyleinterfacesimilar toggplot2, al lowing layered construction of plots.

8. seaborn.axisgrid Purpose: Grid-basedplotting

interfaces. **Key Classes:** - FacetGrid, PairGrid, JointGrid

Details: Createsgridsof subplotsforvisualizing relationshipsacrossmultiple variables and dimensions.

9. seaborn.style Purpose: Controlling plot aesthetics and context.

Key Functions: - set_theme(): Set multiple theme parameters - set_style(): Set the aesthetic style of plots - set_context(): Set the plotting context parameters

Details: Controlstheoverall appearanceof plots, includingcolors, linewidths, and sizes for different contexts (paper, notebook, talk, poster).

10. seaborn.palette Purpose: Color palette utilities.

Key Functions: - color_palette(): Return a list of colors defining a color palette - set_palette(): Set the matplotlib color cycle using a seaborn palette

Details: Providescarefullydesignedcolorpalettesfor categorical andcontinu ous data.

11. seaborn.utils Purpose: Utility functions.

Details: Internal utilities used by seaborn for data processing and plot con struction.

12. seaborn.algorithms Purpose: Statistical algorithms.

Details: Implements statistical algorithms like kernel density estimation and bootstrap confidence intervals.

Plotly

Purpose: Plotly is an interactive graphing library that makes publication quality graphs online. It supports interactive features like hover text, click events, and zooming.

Core Modules

1. plotly.graph_objects Purpose: Low-level interface for creating plots using graph objects.

Key Classes: - Figure: The main figure class - Scatter: Scatter plot trace - Bar: Bar chart trace - Heatmap: Heatmap trace - Scatter3d: 3D scatter plot - Surface: 3D surface plot

Details: Graph objects provide fine-grained control over plot construction. Each trace type has specific parameters for customization. Figures are com posed of traces (data) and layout (appearance).

2. plotly.express Purpose: High-level interface for creating complete fig ures with minimal code.

Key Functions: - scatter(): Create scatter plots - line(): Create line plots - bar(): Create bar charts - histogram(): Create histograms - box(): Create box plots - scatter_3d(): Create 3D scatter plots - choropleth(): Create choropleth maps

Details: Expressfunctionsabstract awaycomplexityandautomaticallycreate completefigureswithproper legends, labels, andcolor scales.Operatesdirectly on DataFrames with minimal code.

3. plotly.subplots Purpose: Creating figures with multiple

subplots. **Key Functions:** - make_subplots(): Create a figure with

subplots

Details: Allows creation of complex multi-panel figures with shared or inde pendent axes. Supports mixing different plot types in the same figure.

4. plotly.io Purpose: Input/output operations for figures.

Key Functions: - write_html(): Write figure to an HTML file - write_image(): Write figure to a static image file - show(): Display figure - to_json(): Convert figure to JSON

Details: Handles figure serialization, file I/O, and integration with different rendering contexts (browser, notebook, static export).

5. plotly.offline Purpose: Tools for offline plotting (deprecated in favor

of plotly.io).

Key Functions: - plot(): Create and open an HTML file with the plot - iplot(): Display plot inline in Jupyter notebook

Details: Originally provided offline plotting capabilities. Most functionality has been moved to plotly.io and the main Figure.show() method.

6. plotly.tools Purpose: Utility tools for plotly (deprecated in favor of plotly.subplots).

Details: Legacy module containing utilities for subplot creation and figure manipulation.

7. plotly.utils Purpose: Utility functions.

Details: Internalutilities forJSON encoding, data validation, and other helper functions.

8. plotly.animation Purpose: Animation utilities.

Details: Providestools for creating animatedplotswithframesandtransition controls.

9. plotly.exceptions Purpose: Custom exceptionclasses.

Details: Defines specific exception types for plotly

operations.

Numerical Computing Libraries

NumPy

Purpose: NumPy is the fundamental package for scientific computing in Python. It provides a powerful N-dimensional array object, sophisticated broadcasting functions, tools for integrating C/C++ code, and useful linear algebra, Fourier transform, and random number capabilities.

Core Modules

1. numpy.core Purpose: Core functionality and ndarray implementation.

Details: Contains the fundamental array object and basic operations. This is where the multi-dimensional array (ndarray) class is defined.

2. numpy.linalg Purpose: Linearalgebra operations.

Key Functions: - det(): Compute matrix determinant - inv(): Compute matrix inverse - eig(): Compute eigenvalues and eigenvectors - solve(): Solve linear equation system - svd(): Singular Value Decomposition - qr(): QR decomposition - cholesky(): Cholesky decomposition - norm(): Matrix or

vector norm

Details: Providesa completesetoflinearalgebra operationsimplementedusing LAPACK for efficiency. Essential for scientific computing, solving systems of equations, and matrix analysis.

3. numpy.fft Purpose: Fast Fourier Transform operations.

Key Functions: - fft(): Compute 1-D discrete Fourier Transform - ifft(): Compute 1-D inverse discrete Fourier Transform - fft2(): Compute 2-D dis crete Fourier Transform - fftn(): Compute N-D discrete Fourier Transform - rfft(): Real input FFT - fftfreq(): Return the sample frequencies

Details: Implements Fast Fourier Transform algorithms for converting between time/spatial domain and frequency domain. Essential for signal processing, spectral analysis, and solving partial differential equations.

4. numpy.random Purpose: Random number generation.

Key Functions: - rand(): Random values in [0, 1) - randn(): Random values from standard normal distribution - randint(): Random integers - choice(): Random sample from array - shuffle(): Shuffle array in-place - seed(): Seed the random number generator

Details: Providesrandomnumbergeneratorsforvariousprobabilitydistribu tions. Essentialfor MonteCarlosimulations, statistical sampling,andmachine learning initialization.

5. numpy.ma Purpose: Masked array operations (arrays with miss ing/invalid entries).

Key Functions: - masked_array(): Create a masked array - masked_where(): Maskwhereconditionis true -filled(): Returnarraywithmaskedvalues filled

Details: Handlesarrayswithmissing or invaliddata points.Themaskindicates which elements are valid/invalid, allowing operations to ignore invalid data.

6. numpy.testing Purpose: Testing utilities.

Key Functions: - assert_array_equal(): Test if two arrays are equal - assert_allclose(): Test if two arrays are equal within tolerance - assert_raises(): Test if function raises expected exception

Details: Providesassertionfunctionsspecificallydesignedfor testingnumerical code withfloating-pointarrays. Handles tolerance for floating-point comparison.

7. numpy.lib Purpose: Additional utilities and functions.

Details: Contains miscellaneous utility functions that don't fit in other cate

gories, including array manipulation, I/O, and type handling.

8. numpy.polynomial Purpose: Polynomial classes andoperations.

Key Classes: -Polynomial: Power series polynomial- Chebyshev: Chebyshev series - Legendre: Legendre series

Details: Provides convenient classes for creating, manipulating, and fitting polynomials. Supports various polynomial bases includingpower series,Cheby shev, Legendre, Laguerre, and Hermite polynomials.

9. numpy.ctypeslib Purpose: Interface for using ctypes to call C code.

Details: Provides utilities for converting between numpy arrays and C data types, facilitating integration with C libraries.

10. numpy.f2py Purpose: Fortran-to-Python interface generator.

Details: Tool for creating Python extensions from Fortran code, allowing Python to call Fortran subroutines.

11. numpy.emath Purpose: Mathematical functions with automatic do main handling.

Details: Provides versions of mathematical functions that return complex re sults when appropriate, rather than raising errors for invalid inputs.

Pandas

Purpose: Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool built on top of Python. It provides data structures like DataFrame and Series that make working with structured data intuitive and efficient.

Core Modules

1. pandas.core Purpose: Core functionality includingDataFrameand Se ries.

Key Classes: -DataFrame: Two-dimensional labeled data structure - Series: One-dimensional labeled array

Details: The heart of pandas, containing the main data structures and their methods. DataFrameislikea spreadsheetwithlabeledrowsandcolumns.Series is like a column in a spreadsheet.

2. pandas.io Purpose: Input/output operations for various file formats.

Key Functions: - read_csv(): Read CSV file - read_excel(): Read Excel file - read_sql(): Read SQL query or database table - read_json(): Read JSON file - read_html(): Read HTML tables - read_parquet(): Read Parquet file - to_csv():

Write to CSV - to_excel(): Write to Excel - to_sql(): Write to SQL database

Details: ProvidescomprehensiveI/Ocapabilities for reading andwritingdata in many formats. Handles data type inference, missing value representation, and format-specific options.

3. pandas.plotting Purpose: Plotting functionality built on matplotlib.

Key Functions: - scatter_matrix(): Matrix of scatter plots - andrews_curves(): Andrews curves for visualizing multivariate data - parallel_coordinates(): Parallel coordinates plot - table(): Create a table plot

Details: Extends DataFrame and Series with plotting capabilities. Provides convenient wrappers around matplotlib for common plotting tasks.

4. pandas.util Purpose: Utility functions.

Details: Contains various utility functions for internal pandas operations and testing.

5. pandas.tseries Purpose: Time series functionality.

Key Classes:- Date offsets and frequencies - Holidaycalendars

Details: Providestoolsforworkingwithtimeseriesdata, includingdateoffsets, frequency strings, and holiday calendars.

6. pandas.sparse Purpose: Sparsedata structures(deprecated, usepandas arrays instead).

Details: Originally provided sparse versions of Series and DataFrame for memory-efficient storage of sparse data.

7. pandas.arrays Purpose: Extension array types.

Key Classes: - IntegerArray: Integer array with NA support - StringArray: String array - BooleanArray: Boolean array with NA support - DatetimeArray: Datetime array - TimedeltaArray: Timedelta array - IntervalArray: Interval array - PeriodArray: Period array - ArrowExtensionArray: PyArrow-backed arrays

Details: Provides specialized array types that extend NumPy's type system. Thesesupportpandas-specific featureslikenullableintegertypesandcategorical data.

8. pandas.api Purpose: Public API for extensions.

Details: Contains the public API for creating custom extension types and integrating with pandas.

9. pandas.errors Purpose: Custom exception classes.

Details: Defines pandas-specific exception types for better error handling.

10. pandas.testing Purpose: Testing utilities.

Key Functions: - assert_frame_equal(): Test if two DataFrames are equal - assert_series_equal(): Test if two Series are equal

Details: Providesassertionfunctionsfor testingpandascode, handling floating point tolerance and data type comparison.

11. pandas.compat Purpose: Compatibility utilities.

Details: Provides compatibility layers for different Python and NumPy ver sions.

SciPy

Purpose: SciPy is a collection of mathematical algorithms and convenience functionsbuiltonNumPy.Itprovidesmodulesforoptimization,linearalgebra, integration, interpolation, specialfunctions, FFT, signal andimageprocessing, ODE solvers, and more.

Core Modules

1. scipy.cluster Purpose: Clustering algorithms.

Key Functions: - scipy.cluster.hierarchy: Hierarchical clustering - scipy.cluster.vq: Vector quantization / K-means

Details: Provides algorithms for clustering data points and creating dendro grams for hierarchical clustering.

2. scipy.constants Purpose: Physical and mathematical constants.

Key Constants: - pi, golden: Mathematical constants - c: Speed of light - h: Planck's constant - e: Elementary charge

Details: Providesa collectionof physical constantsandconversionfactorswith high precision.

3. scipy.fft Purpose: Fast Fourier Transform functions.

Details: ImplementsFFTalgorithms, similartonumpy.fftbutwithadditional features and optimizations.

4. scipy.integrate Purpose: Integrationandordinarydifferential equation solvers.

Key Functions: - quad(): General purpose integration - dblquad(), tplquad(): Double and triple integrals - odeint(): Solve ordinary differential equations (legacy) - solve_ivp(): Solve initial value problems for ODEs

(newer interface)

Details: Providestoolsfornumericalintegrationoffunctionsandsolving ordi narydifferential equations.Essentialforphysicssimulations, engineering calcu lations, and scientific computing.

5. scipy.interpolate Purpose: Interpolation and smoothing of data.

Key Functions: - interp1d(): 1-D interpolation - griddata(): Interpo late unstructured D-D data - CubicSpline(): Cubic spline interpolation - RBFInterpolator(): Radial basis function interpolation - interpn(): Mul tidimensional interpolation on regular grids

Details: Providesmethodsfor estimatingunknownvaluesbetweenknowndata points. Essential for resampling data, image processing, and reconstructing continuous functions from discrete samples.

6. scipy.io Purpose: Input and output operations.

Key Functions: - loadmat(): Load MATLAB file - savemat(): Save data to MATLAB file - wavfile.read(): Read WAV file - wavfile.write(): Write WAV file

Details: Handlesreadingandwritingdata invariousfileformats, particularly MATLAB files and audio files.

7. scipy.linalg Purpose: Linear algebra operations (extended version of numpy.linalg).

Key Functions: -All functions from numpy.linalg plus: - expm(): Matrix ex ponential - logm(): Matrix logarithm - sqrtm(): Matrix square root - schur(): Schur decomposition -lu(): LU decomposition

Details: Provides more comprehensive linear algebra operations than NumPy, with additional decompositions and matrix functions. Always uses BLAS/LAPACK implementations.

8. scipy.ndimage Purpose: Multi-dimensional image processing.

Key Functions: - gaussian_filter(): Gaussian filtering - median_filter(): Median filtering - rotate(): Rotate an image - zoom(): Zoom an image - binary_erosion(), binary_dilation(): Morphological operations - label(): Label connected components

Details: Provides a comprehensive set of image processing operations including filtering, morphology, interpolation, measurements, andsegmentation.Works on N-dimensional arrays, not just 2D images.

9. scipy.odr Purpose: Orthogonal distance regression.

Details: Implements ODR(OrthogonalDistance Regression) whichfits a func

tion to data with errors in both x and y variables.

10. scipy.optimize Purpose: Optimizationandroot-finding algorithms.

Key Functions: - minimize(): Minimize a scalar function - minimize_scalar(): Minimize a scalar function of one variable - root(): Find roots of a vector func tion - root_scalar(): Find root of a scalar function - curve_fit(): Fit a func tion to data - linprog(): Linear programming - differential_evolution(): Global optimization

Details: Provides comprehensive optimization capabilities for finding min ima/maxima of functions, solving equations, and fitting models to data. Includes both local and global optimization algorithms.

11. scipy.signal Purpose: Signal processing tools.

Key Functions: - find_peaks(): Find peaks in a signal - welch(): Power spectral density estimation - spectrogram(): Compute spectrogram - butter(), cheby1(): Filter design - filtfilt(): Zero-phase filtering - convolve(): Convolution of two arrays - correlate(): Cross-correlation - stft(): Short-time Fourier transform

Details: Comprehensive signal processing functionality including filter design, spectral analysis,waveform generation, andpeak detection. Essential fordigital signal processing, audio processing, and time series analysis.

12. scipy.sparse Purpose: Sparse matrix operations.

Key Classes: - csr_matrix: Compressed Sparse Row matrix - csc_matrix: Compressed Sparse Column matrix - coo_matrix: COOrdinate format matrix - lil_matrix: List of Lists format matrix

Details: Provides efficient storage and operations for sparse matrices (matri ces with mostly zero elements). Essential for large-scale linear algebra, graph algorithms, and machine learning with high-dimensional data.

13. scipy.spatial Purpose: Spatial data structures andalgorithms.

Key Functions: - distance.cdist(): Pairwise distances between points - KDTree: k-d tree for fast nearest neighbor search - Delaunay: Delaunay trian gulation - Voronoi: Voronoi diagrams - ConvexHull: Convex hull computation

Details: Provides algorithms for spatial computations including distance cal culations, nearest neighbor search,triangulation, andcomputational geometry.

14. scipy.special Purpose: Special mathematical functions.

Key Functions: - gamma(): Gamma function - erf(): Error function - bessel(): Bessel functions - beta(): Beta function - factorial(): Factorial

Details: Provides a large collection of special functions used in mathematical physics, statistics, and engineering.

15. scipy.stats Purpose: Statistical functions and probability distributions.

Key Features: - Probability distributions (normal, t, chi-square, etc.) - Sta tistical tests (t-test, chi-square test, ANOVA, etc.) - describe(): Descriptive statistics - pearsonr(): Pearson correlation coefficient - spearmanr(): Spear man correlation coefficient - ks_2samp(): Kolmogorov-Smirnov test

Details: Comprehensive statistical functionality including 100+ continuous and discrete probability distributions, statisticaltests, descriptive statistics, and correlation functions. Essential for statistical analysis and hypothesis testing.

Hyperparameter Tuning Packages

While sklearn.model_selection provides GridSearchCV (testing every combination) and RandomizedSearchCV (testing random combinations), these methods can be inefficient. The libraries below offer more advanced, intelligent, and scalable search strategies.

Installation: pip install optuna hyperopt "ray[tune]"

1. Optuna

Standard Alias:import optuna

Purpose: A modern, powerful, and easy-to-use hyperparameter optimization framework. It is framework-agnostic (works with sklearn, PyTorch, etc.) and features an intuitive API.

Key Concepts:

- **Study:** The main object that manages an optimization session. You create a study and tell it to maximize or minimize a metric.
- **Trial:** A single execution of your model with a specific set of hyperparameters. Inside your objective function, you ask the trial object to "suggest" values (e.g., trial.suggest_float, trial.suggest_int, trial.suggest_categorical).
- **Pruning:** Optuna can automatically stop unpromising trials early (e.g., after a few epochs of a neural net), saving massive amounts of time.
- **Visualization:** Provides built-in functions to visualize optimization history, parameter importance, and more.

2. Hyperopt

Standard Alias:from hyperopt import fmin, tpe, hp, Trials

Purpose: A popular and established library for serial and parallel optimization. It is well-known for its implementation of **TPE (Tree-structured Parzen Estimator)**, a Bayesian optimization algorithm that intelligently chooses the next set of hyperparameters based on past results.

Key Concepts:

- **Search Space:** Defined using hp objects (e.g., hp.choice, hp.uniform, hp.lognormal). This is where you define the range of possible values for your hyperparameters.
- **fmin:** The main function that runs the optimization. You provide it with your objective function, the search space, the search algorithm (like tpe.suggest), and the max number of evaluations.
- **Trials:**An object that stores the history of all trials, which you can later inspect.

3. Ray Tune (from Ray)

Standard Alias:from ray import tune

Purpose: A scalable hyperparameter tuning library built for speed and distribution. It is part of the larger ray ecosystem, which is designed for parallel and distributed Python. Use Ray Tune when your tuning jobs are very large, you need to run on multiple machines (a cluster), or you need advanced scheduling.

Key Concepts:

- **Scalability:**Its primary feature. Can easily scale a tuning job from your laptop to a large cluster.
- **Schedulers:** Advanced algorithms for stopping trials early (like ASHA, PBT) that are more powerful than basic pruning.
- **Search Algorithms:**Integrates with many other libraries, allowing you to use Optuna, Hyperopt, and others as its search algorithm, but in a distributed way.